

Deferred Database Modification

Log-Based Recovery, No-UNDO / REDO Strategy & Comparative Analysis

1. WHAT IS LOG-BASED RECOVERY?

Log-based recovery is a technique used to recover database consistency after a system crash or failure. The transaction log records all execution events, allowing the recovery manager to decide whether to re-apply or discard operations upon restart.

2. WHAT IS DEFERRED DATABASE MODIFICATION?

In Deferred Database Modification, changes made by a transaction are **not written to the actual database on disk immediately**. Instead, modifications are buffered in memory/RAM and are written to disk only **after** the transaction commits.

★ Golden Rule

Deferred = Database Update Delayed Until COMMIT. This avoids writing uncommitted data to disk and eliminates the need for rolling back (UNDOing) aborted transactions.

3. TRANSACTION EXECUTION & MEMORY STATE

Consider initial database values on disk: $A = 100$, $B = 200$. Transaction T_1 executes updates:

RAM / MEMORY STATE

READ(A) $\rightarrow A = A + 100 \rightarrow$ WRITE(A) ($A = 200$)

READ(B) $\rightarrow B = B + 200 \rightarrow$ WRITE(B) ($B = 400$)

RAM holds new values: $A = 200$, $B = 400$

DISK / DB STATE

Before COMMIT: Disk values remain untouched!

$A = 100$, $B = 200$

After COMMIT: Updates written to disk:

$A = 200$, $B = 400$

4. TRANSACTION LOG FORMAT

Because disk values remain original prior to commit, the log **only records New Values (V_{new})**. Old values are not stored.

```
<T1 START>
<T1, A, 200> <-- <Txn_ID, Data_Item, New_Value>
<T1, B, 400>
<T1 COMMIT>
```

5. RECOVERY ACTION ON SYSTEM CRASH

Case A: Failure AFTER Commit

Log contains both `<T1 START>` and `<T1 COMMIT>`.

Action: REDO T1

Applies new values ($A = 200$, $B = 400$) to disk to ensure permanence.

Case B: Failure BEFORE Commit

Log contains `<T1 START>` but **NO COMMIT**.

Action: Do Nothing / Discard

Disk was never modified (still $A=100$, $B=200$). No UNDO needed!

Deferred Database Modification = No-UNDO / REDO Strategy

6. MULTI-TRANSACTION CRASH SCENARIO

Consider the following log state when a crash occurs:

```
<T1 START>
<T1, A, 200>
<T1 COMMIT>
<T2 START>
<T2, B, 500>
💣 SYSTEM FAILURE
```

RECOVERY EVALUATION

- Transaction T_1 : Log has **START** + **COMMIT**.
→ Action: **REDO**(T_1) (Set $A = 200$)
- Transaction T_2 : Log has **START** but **NO COMMIT**.
→ Action: **Discard** T_2 (DB retains prior $B = 200$)

7. COMPARATIVE ANALYSIS: DEFERRED VS. IMMEDIATE MODIFICATION

FEATURE	DEFERRED MODIFICATION	IMMEDIATE MODIFICATION
DB Updated Before Commit?	❌ No (Only in RAM)	✅ Yes (Direct to disk)
DB Updated After Commit?	✅ Yes	Already updated during execution
Log Stores Old Values?	❌ No (Not required)	✅ Yes (Old + New)
Log Stores New Values?	✅ Yes	✅ Yes
UNDO Required?	❌ No	✅ Yes
REDO Required?	✅ Yes	✅ Yes
Strategy Name	No-UNDO / REDO	UNDO / REDO

8. HIGH-YIELD EXAM Q&A ⚠️

Q1: Why are Old Values unnecessary?

Because disk records are never overwritten before commit, original values remain preserved on disk natively.

Q2: Clarifying "No-UNDO / REDO"

"No-UNDO / REDO" means **UNDO is omitted**, but **REDO is strictly required** for committed transactions.

9. ULTIMATE MEMORY TRICK & 10-SECOND REVISION

⚡ 10-SECOND REVISION CHEATSHEET

Delay DB Update → Wait for COMMIT → Log stores NEW Values Only

↓

On Crash:

Committed → REDO | Uncommitted → Do Nothing (Discard)

🔑 **Deferred = Update Later + New Values Only + No-UNDO / REDO**